1/15/2026

Machine Learning -2

Project-5

Tushar Choudhury

Contents

List of Figures 3

Exploratory Data Analysis 5

- ***Data Overview*** 5
- ***Information About Data*** 5
- ***Description of Data*** 6
- ***Univariate Analysis*** 7
  - ***Observations on case_status*** 7
  - ***Observations on continent*** 7
  - ***Observations of education_0f_employee*** 8
  - ***Observations on has_job_experience*** 9
  - ***Observations on requires_job_training*** 9
  - ***Observations on no_of_employees*** 10
  - ***Observations on yr_of_estab*** 10
  - ***Observations on region_of_employment*** 11
  - ***Observations on prevailing_wage*** 12
  - ***Observations of unit_of_wage*** 12
  - ***Observations on full_time_position*** 13
- ***Bivariate Analysis*** 13
  - ***Observations between case-status and continent*** 13
  - ***Observations between education_of_employees and case_status*** 14
  - ***Observations between has_job_experience and case_status*** 14
  - ***Observations between region_of_employment and case_status*** 15
  - ***Observations between unit_of_wage and case_status*** 15
  - ***Observations between no_of_employees and case_status*** 16
  - ***Heatmap*** 17

Data Preprocessing 17

- ***Missing value Treatment*** 17
- ***Outliers Check*** 18
- ***Data Preparation for modelling*** 18
- ***Data Splitting*** 19

Model Building 20

- ***Model Building - Original Data*** 20
- ***Model Building - Oversampled Data*** 21
- ***Model Building - Undersampled Data*** 22

Hyperparameter Tuning 23

Tuning AdaBoostClassifier model with Undersampled data 23
Finding Best parameters 23
Model tuning with best parameters 23
Training performance of model with best parameters 24
Validation performance of model with best parameters 24
Tuning Gradient Boosting model with Undersampled Data 25
Finding Best Parameters 25
Model tuning with best parameters 25
Training performance of model with best parameters 26
Validation performance of model with best parameters 26
Tuning Adaboosting model with Oversampled data 27
Finding Best Parameters 27
Model tuning with best parameters 27
Training performance of model with best parameters 28
Validation performance of model with best parameters 28
Tuning Gradient Boosting model with Oversampled data 29
Finding Best Model 29
Model tuning with best parameters 29
Training performance of model with best parameters 30
Validation performance of model with best parameters 30
Model Comparison and Final Model Selection 31
Training performance comparison 31
Validation performance comparison 31
Feature Importance 32
Performance on test set 32

List of Figures

Figure 1 – Data 5
Figure 2 - Information about Data 5
Figure 3 - Description of Data 6
Figure 4 - Negative values check 6
Figure 5 - Labelled Barplot of case_status column 7
Figure 6 - Labelled Barplot of continent column 7
Figure 7 - Labelled Barplot of education_of_employee column 8
Figure 8 - Labelled Barplot of has_job_experience column 9
Figure 9 - Labelled Barplot of requires_job_training column 9
Figure 10 - Histogram_Boxplot of no_of_employees column 10
Figure 11 - Histogram_boxplot of yr_of_estab column 10
Figure 12 - Labelled Barplot of region_of_employment column 11
Figure 13 - Histogram Boxplot of prevaliing_wage 12
Figure 14 - Labelled Barplot of unit_of_wage 12
Figure 15 - Labelled Barplot of full_time_position column 13
Figure 16 - Stacked Barplot between case_status and continent 13
Figure 17 - Stacked Barplot between education_of_employee and case_status 14
Figure 18 - Stacked barplot between has_job_experience and case_status 14
Figure 19 - Stacked barplot between region_of_employment and case_status 15
Figure 20 - Stacked barplot between unit_of_wage and case_status 15
Figure 21 - Distribution plot between no_of_employees and case_study 16
Figure 22 - Heatmap of numerical columns 17
Figure 23 - Null values count 17
Figure 24 - Duplicate value count 18
Figure 25 - Outliers of columns 18
Figure 26 - Data after pre-processing 19
Figure 27 - Information of Data after pre-processing 19
Figure 28 - Train-Test-Split 19
Figure 29 - Recall Scores of 5 different model performance on original Data 20
Figure 30 - Performance difference of model on Training data and Validation data 20
Figure 31 - Oversampling of data 21
Figure 32 - Recall scores of 5 different model performance on oversampled data 21
Figure 33 - Performance difference of Oversampled model on Training data and Validation data 21
Figure 34 - Under sampling of data 22
Figure 35 - Recall scores of 5 different model performance on Undersampled data 22
Figure 36 - Performance difference of Undersampled model on Training data and Validation data 22
Figure 37 - Hyper parameter tuning of Ada boost classifier with undersampled data 23
Figure 38 - Best parameters 23
Figure 39 - Model with best parameters 23
Figure 40 - Performance matrices of training set 24
Figure 41 - confusion matrix of training set 24
Figure 42 - Performance matrices of validation set 24
Figure 43 - Confusion matrix of validation set 24
Figure 44 - Hyper parameter tuning of Gradient boost classifier with undersampled data 25
Figure 45 - Best parameters 25

Figure 46 - Best Model 25
Figure 47 - Performance metrices of training set 26
Figure 48 - confusion matrix of training set 26
Figure 49 - Performance metrices of validation set 26
Figure 50 - confusion matrix of validation set 26
Figure 51 - Hyperparameter tuning of Ada boost model with Oversampled data 27
Figure 52 - Best Parameters 27
Figure 53 - Best Model 27
Figure 54 - Performance matrices of training set 28
Figure 55 - confusion matrix of training set 28
Figure 56 - Performance matrices of validation set 28
Figure 57 - confusion matrix of validation set 28
Figure 58 - Hyperparameter tuning of Gradient boosting model with Oversampled Data 29
Figure 59 - Best Parameters 29
Figure 60 - Best Model 29
Figure 61 - Performance matrices of training set 30
Figure 62 - confusion matrix of training set 30
Figure 63 - Performance matrices of validation set 30
Figure 64 - confusion matrix of validation set 30
Figure 65 - Performance comparison of Training set 31
Figure 66 - Performance comparison of Validation set 31
Figure 67 - Feature Importances on final model 32
Figure 68 - Performance metrices of testing set 32
Figure 69 - Confusion matrix of testing set 33

Exploratory Data Analysis

Data Overview

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab
0	EZYV01	Asia	High School	N	N	14513	2007
1	EZYV02	Asia	Master's	Y	N	2412	2002
2	EZYV03	Asia	Bachelor's	N	Y	44444	2008
3	EZYV04	Asia	Bachelor's	N	N	98	1897
4	EZYV05	Africa	Master's	Y	N	1082	2005
...	...	...	...	...	...	...	...
25475	EZYV25476	Asia	Bachelor's	Y	Y	2601	2008
25476	EZYV25477	Asia	High School	Y	N	3274	2006
25477	EZYV25478	Asia	Master's	Y	N	1121	1910
25478	EZYV25479	Asia	Master's	Y	Y	1918	1887
25479	EZYV25480	Asia	Bachelor's	Y	N	3195	1960

25480 rows × 12 columns

Figure 1 – Data

The data Consists of 25480 rows and 12 columns.

Information About Data

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 12 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   case_id                25480 non-null  object
 1   continent              25480 non-null  object
 2   education_of_employee  25480 non-null  object
 3   has_job_experience     25480 non-null  object
 4   requires_job_training  25480 non-null  object
 5   no_of_employees        25480 non-null  int64
 6   yr_of_estab            25480 non-null  int64
 7   region_of_employment   25480 non-null  object
 8   prevailing_wage        25480 non-null  float64
 9   unit_of_wage           25480 non-null  object
 10  full_time_position     25480 non-null  object
 11  case_status            25480 non-null  object
dtypes: float64(1), int64(2), object(9)
memory usage: 2.3+ MB
```

Figure 2 - Information about Data

- *Majority columns in the dataset are **Object** Data types.*
- *The dataset contains **2 int64 dtype, 1 float64 dtype, 9 object dtype.***

Description of Data

	count	mean	std	min	25%	50%	75%	max
no_of_employees	25480.0	5667.043210	22877.928848	-26.0000	1022.00	2109.00	3504.0000	602069.00
yr_of_estab	25480.0	1979.409929	42.366929	1800.0000	1976.00	1997.00	2005.0000	2016.00
prevailing_wage	25480.0	74455.814592	52815.942327	2.1367	34015.48	70308.21	107735.5125	319210.27

Figure 3 - Description of Data

- *There are few errors in **no_of_employees** column.*
- ***no_of_employees** can never be **negative** but there are few negative values which need to be corrected.*

	case_id	continent	education_of_employee	has_job_experience	requires_job_training	no_of_employees	yr_of_estab	r
245	EZYV246	Europe	Master's	N	N	-25	1980	
378	EZYV379	Asia	Bachelor's	N	Y	-11	2011	
832	EZYV833	South America	Master's	Y	N	-17	2002	
2918	EZYV2919	Asia	Master's	Y	N	-26	2005	
6439	EZYV6440	Asia	Bachelor's	N	N	-14	2013	
6634	EZYV6635	Asia	Bachelor's	Y	N	-26	1923	
7224	EZYV7225	Europe	Doctorate	N	N	-25	1998	
7281	EZYV7282	Asia	High School	N	N	-14	2000	
7318	EZYV7319	Asia	Bachelor's	Y	Y	-26	2006	
7761	EZYV7762	Asia	Master's	N	N	-11	2009	
9872	EZYV9873	Europe	Master's	Y	N	-26	1996	
11493	EZYV11494	Asia	High School	Y	N	-14	1999	
13471	EZYV13472	North America	Master's	N	N	-17	2003	

Figure 4 - Negative values check

- *The are 33 negative values in no_of_employees column which are considered as error.*

Univariate Analysis

Observations on case_status

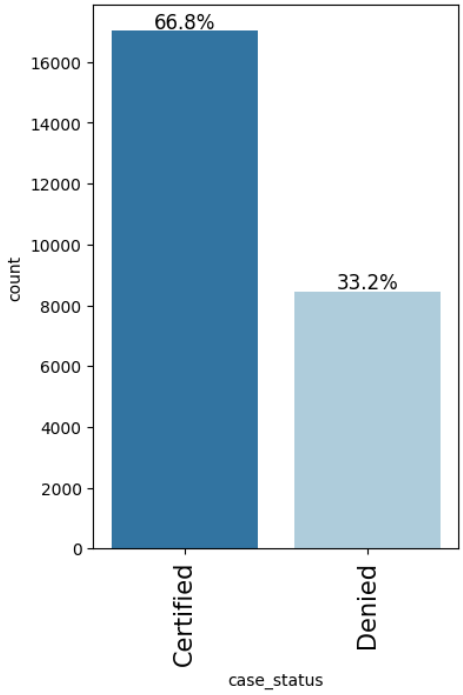

Figure 5 - Labelled Barplot of case_status column

- ***66.8%*** *of employees got their* ***Visa certified.***
- ***33.2%*** *of employees got their* ***Visa denied.***

Observations on continent

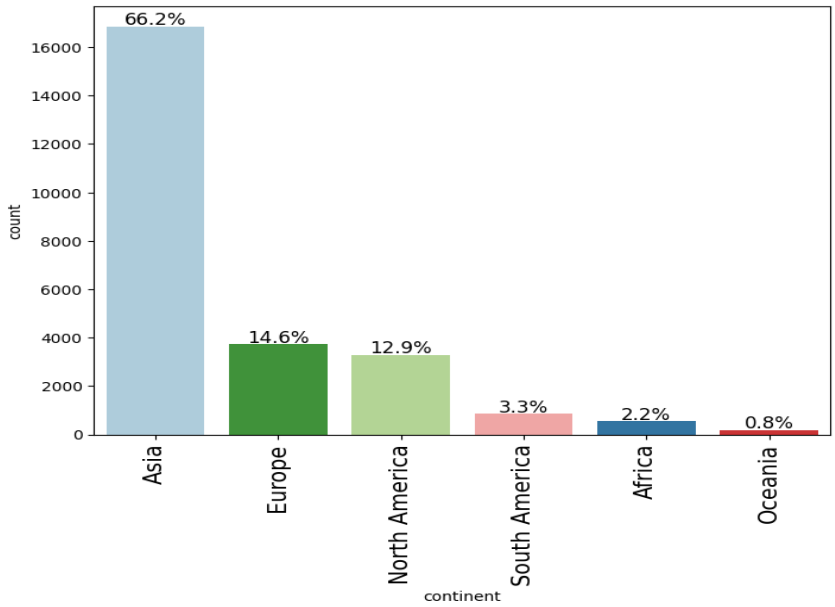

Figure 6 - Labelled Barplot of continent column

- *Most of the employees are from **Asia with 66.2%**.*
- ***14.6%** of employees are from **Europe**.*
- ***12.9%** of employees are from **North America**.*
- ***3.3%** of employees are from **South America**.*
- ***2.2%** of employees are from **Africa**.*
- ***0.8%** of employees are from **Oceania**.*

Observations of education_0f_employee

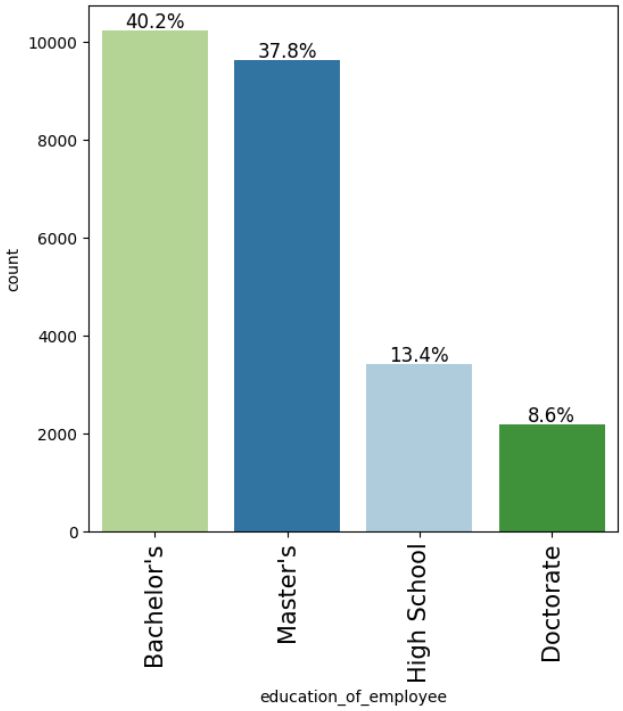

Figure 7 - Labelled Barplot of education_of_employee column

- ***40.2%** of employees have completed their **Bachelor's**.*
- ***37.8%** of employees have completed their **Master's**.*
- ***13.4%** of employees have completed their **High School Education**.*
- ***8.6%** of employees have completed their **Doctorate**.*

Observations on has_job_experience

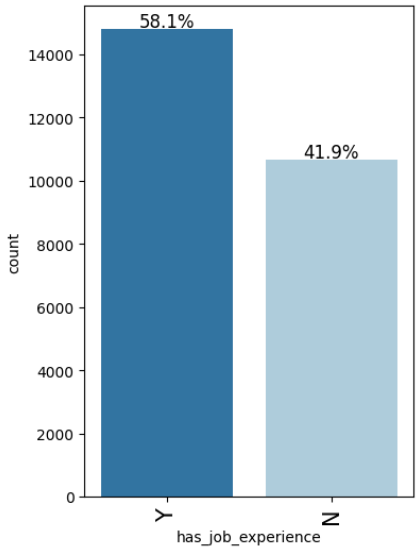

Figure 8 - Labelled Barplot of has_job_experience column

- ***58.1%*** *of employees have prior job experience.*
- ***41.9*** *of employees do not have prior job experience.*

Observations on requires_job_training

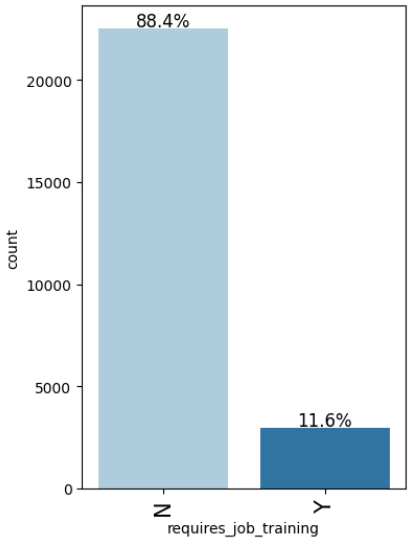

Figure 9 - Labelled Barplot of requires_job_training column

- ***88.4%*** *of employees requires job training.*
- ***11.6%*** *of employees do not require job training.*

Observations on no_of_employees

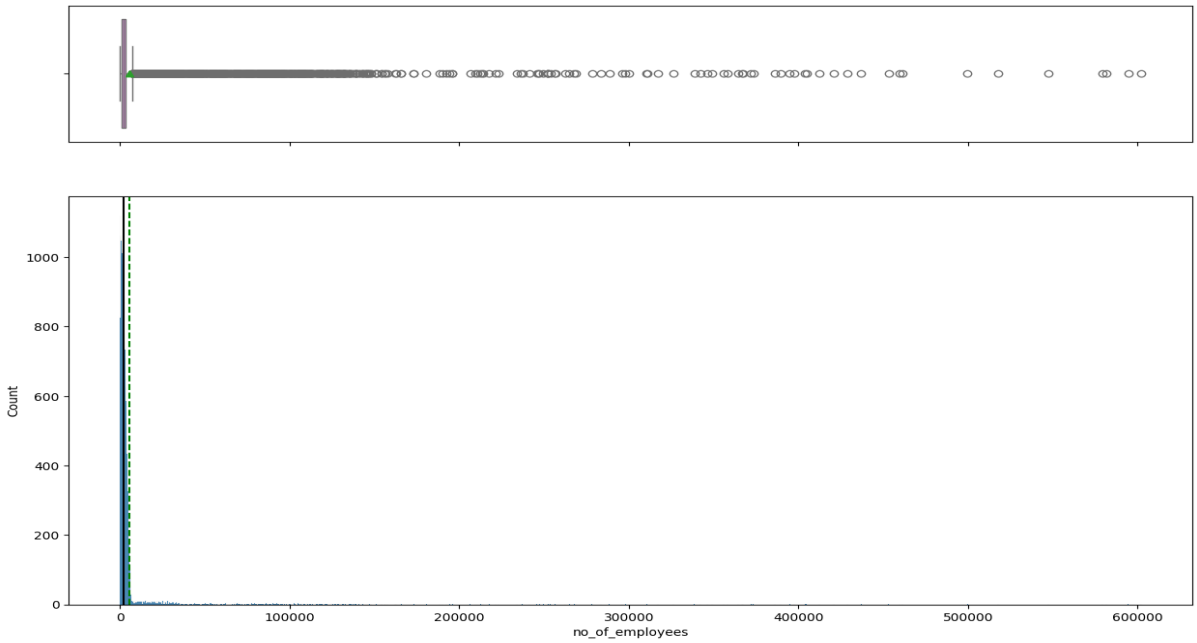

Figure 10 - Histogram_Boxplot of no_of_employees column

- ***602069*** *is the* ***maximum*** *number of employees working in a single company.*
- ***Average*** *number of employees working in each company is* ***5667****.*
- *The histogram of no_of_employees column is* ***right-skewed****.*

Observations on yr_of_estab

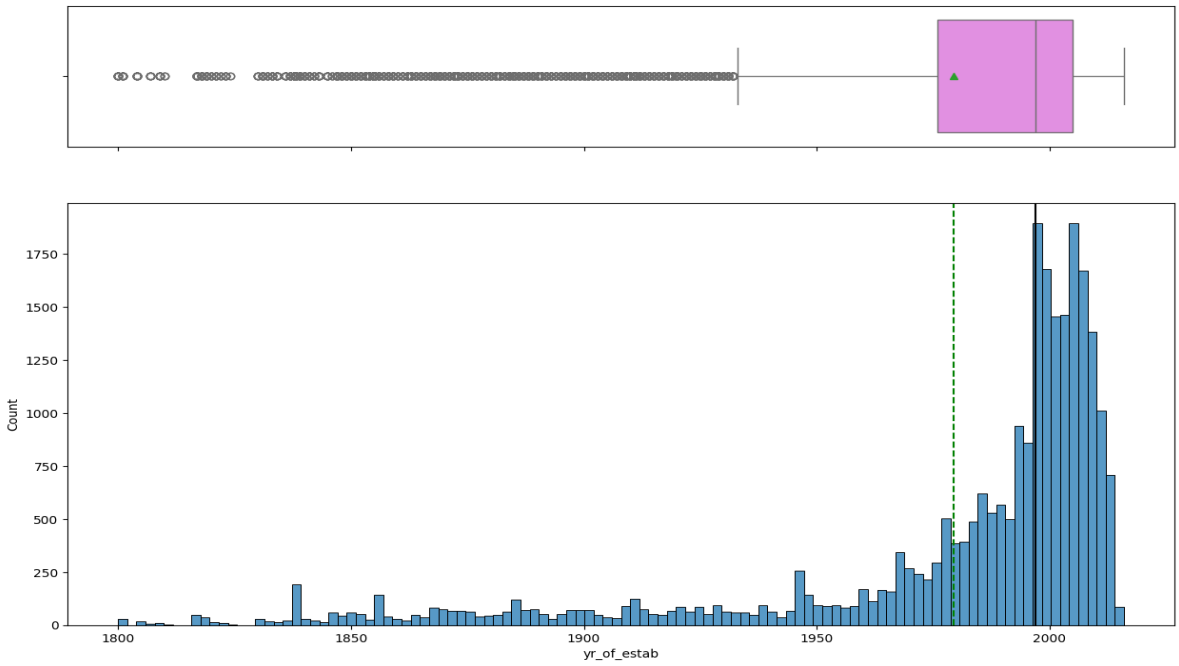

Figure 11 - Histogram_boxplot of yr_of_estab column

- *Most of the companies are established in 1997.*
- *The histogram of yr_of_estab column is **left-skewed**.*

Observations on region_of_employment

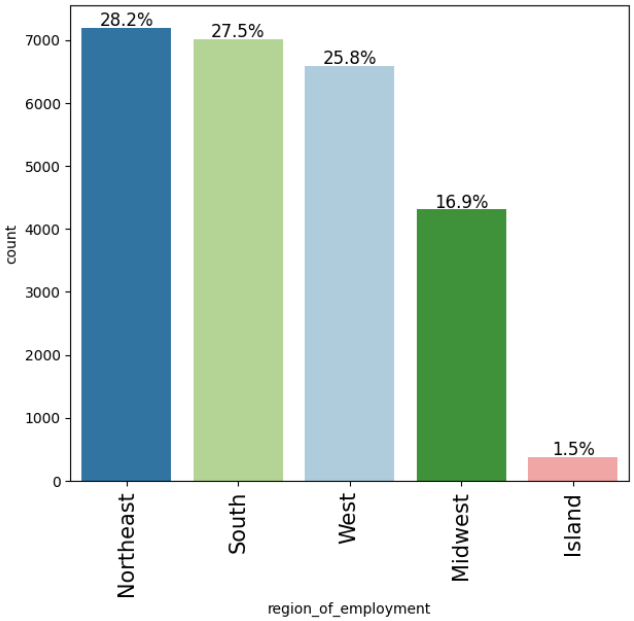

Figure 12 - Labelled Barplot of region_of_employment column

- *Most of the foreign workers are from **Northeast** with a percentage of **28.3%**.*
- ***27.5%** of foreign workers are from the **south**.*
- ***25.8%** of foreign workers are from **west**.*
- ***16.9%** of foreign workers are from **Midwest**.*
- ***1.5%** of foreign workers are from **islands**.*

Observations on prevailing_wage

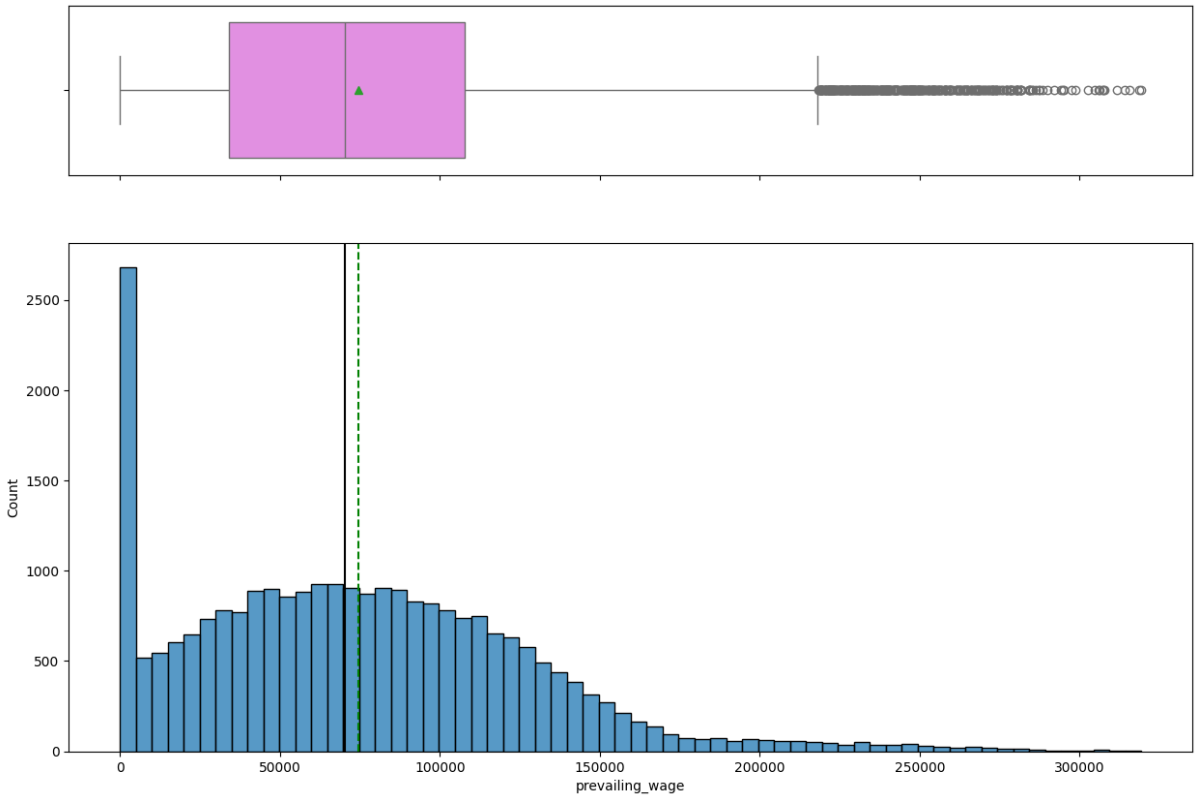

Figure 13 - Histogram Boxplot of prevaliing_wage

- *Average wage paid to employees is **74455$**.*
- *Majority of employees are paid an average wage of **70308$**.*

Observations of unit_of_wage

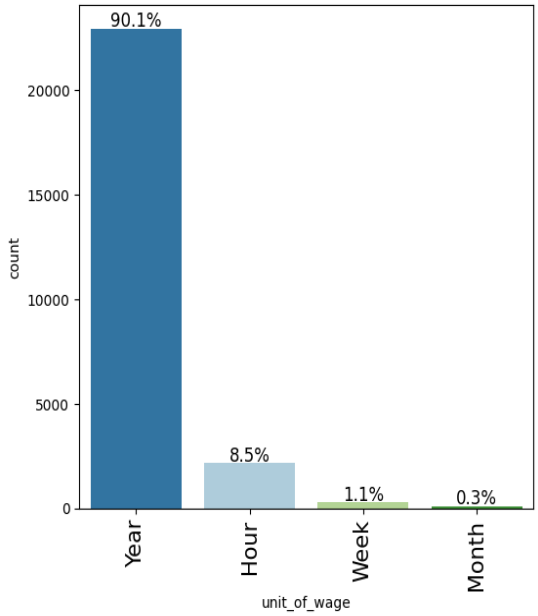

Figure 14 - Labelled Barplot of unit_of_wage

- *Most of the employees are paid on **Yearly basis** with a percentage of **90.1%**.*
- ***8.5%** of employees are paid on **Hourly basis**.*
- ***1.1%** of employees are paid on **Weekly basis**.*
- ***0.3%** of employees are paid on **Monthly basis**.*

Observations on full_time_position

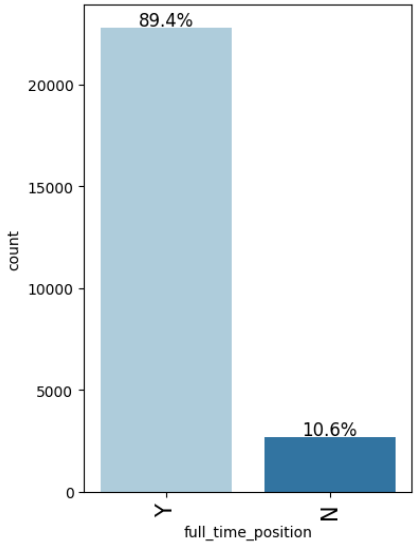

Figure 15 - Labelled Barplot of full_time_position column

- ***89.4%** of employees acquired full time positions.*
- ***10.6%** of employees did not acquire full time positions.*

Bivariate Analysis

Observations between case-status and continent

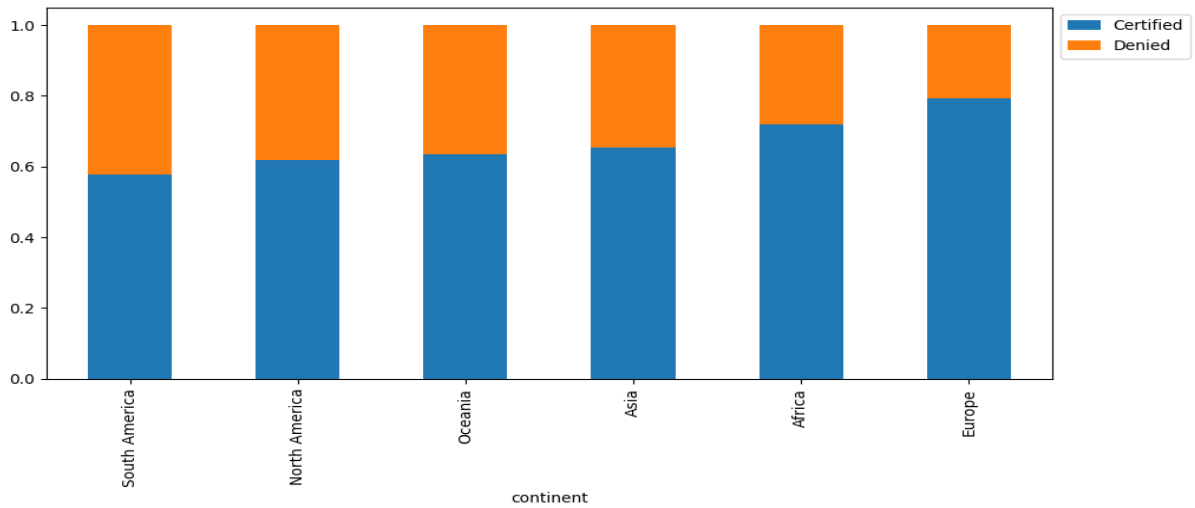

Figure 16 - Stacked Barplot between case_status and continent

- ***South Africa*** *has the most Visa denied applications.*
- ***Europe*** *has the most visa approved applications.*

Observations between education_of_employees and case_status

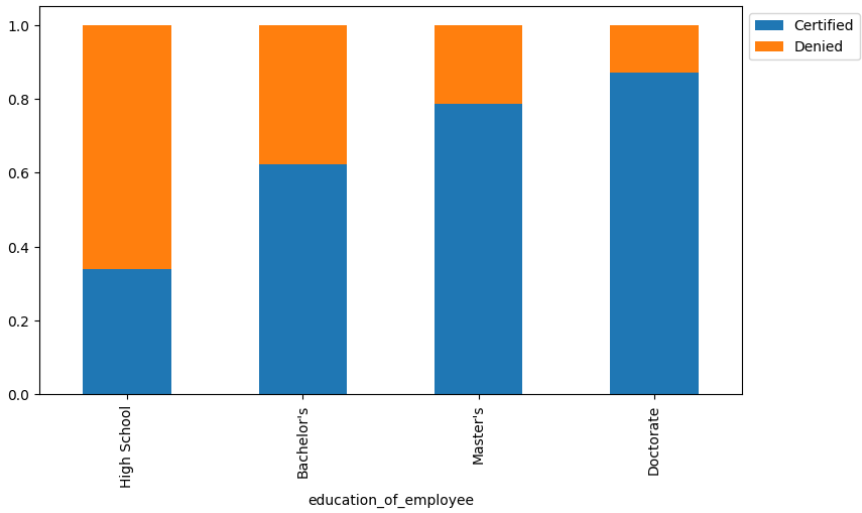

Figure 17 - Stacked Barplot between education_of_employee and case_status

- *Employees with* ***High school graduation*** *have highest visa rejected applications.*
- *Employees with* ***Doctorate*** *have least visa rejection rate.*

Observations between has_job_experience and case_status

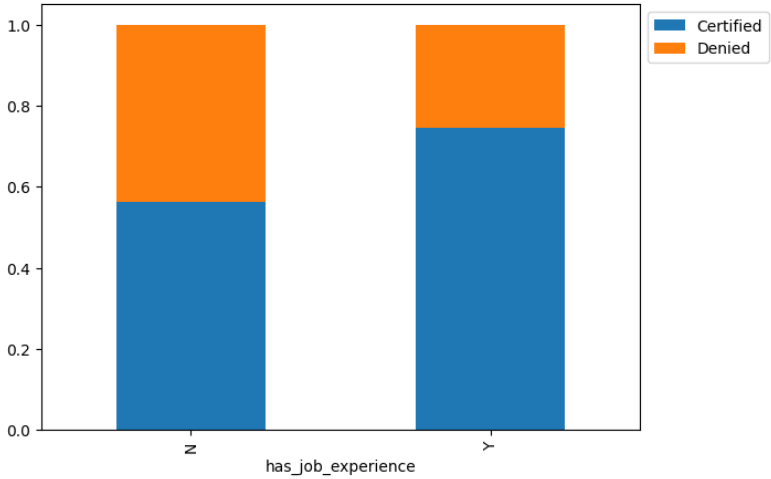

Figure 18 - Stacked barplot between has_job_experience and case_status

- *Employees with job experience have least visa rejection rate.*
- *Employees without job experience have highest visa rejection rate.*

Observations between region_of_employment and case_status

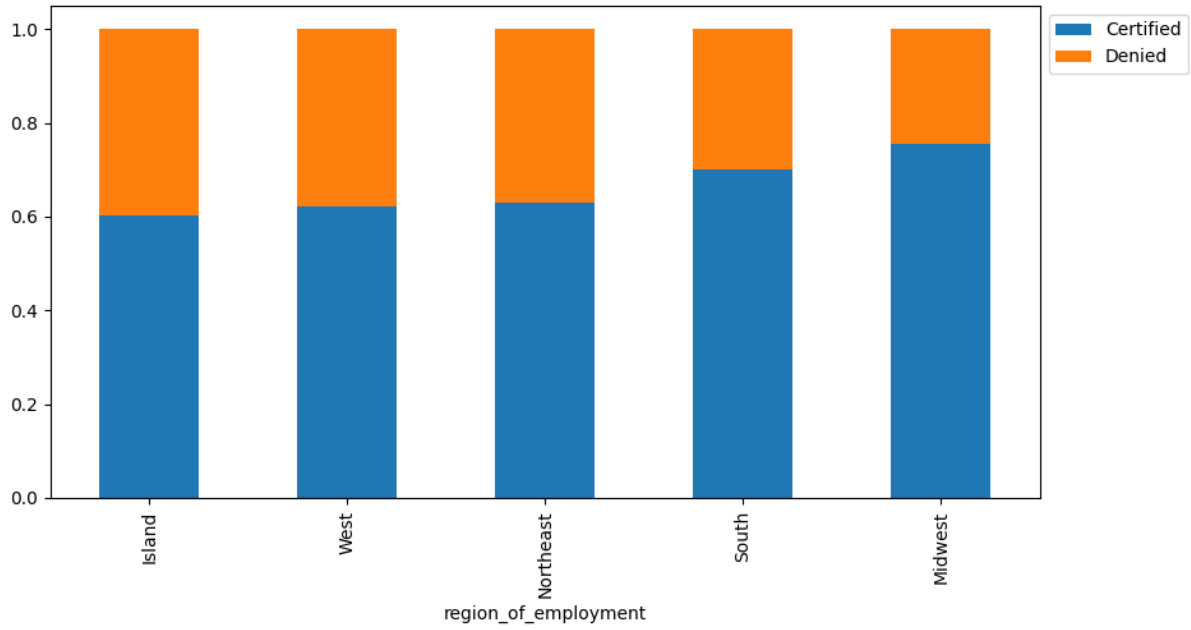

Figure 19 - Stacked barplot between region_of_employment and case_status

- *Employees working in **islands** have highest visa rejection rate.*
- *Employees working in **Midwest** have least visa rejection rate.*
- *Employees working in **west** and **Northeast** have approximately same visa rejection rate.*

Observations between unit_of_wage and case_status

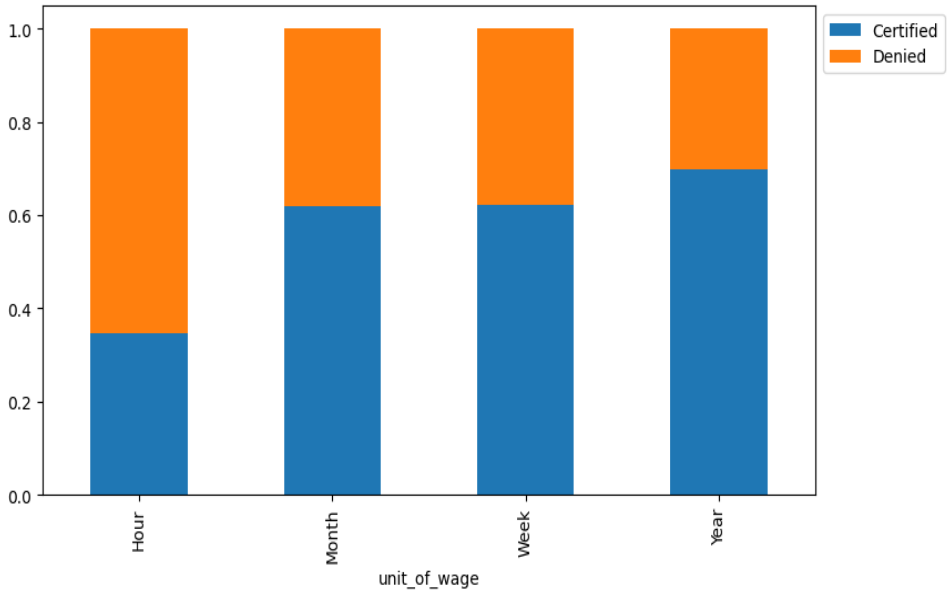

Figure 20 - Stacked barplot between unit_of_wage and case_status

- ***Hourly wage*** *employees have highest visa rejection rate.*
- ***Yearly wage*** *employees have least visa rejection rate.*

Observations between no_of_employees and case_status

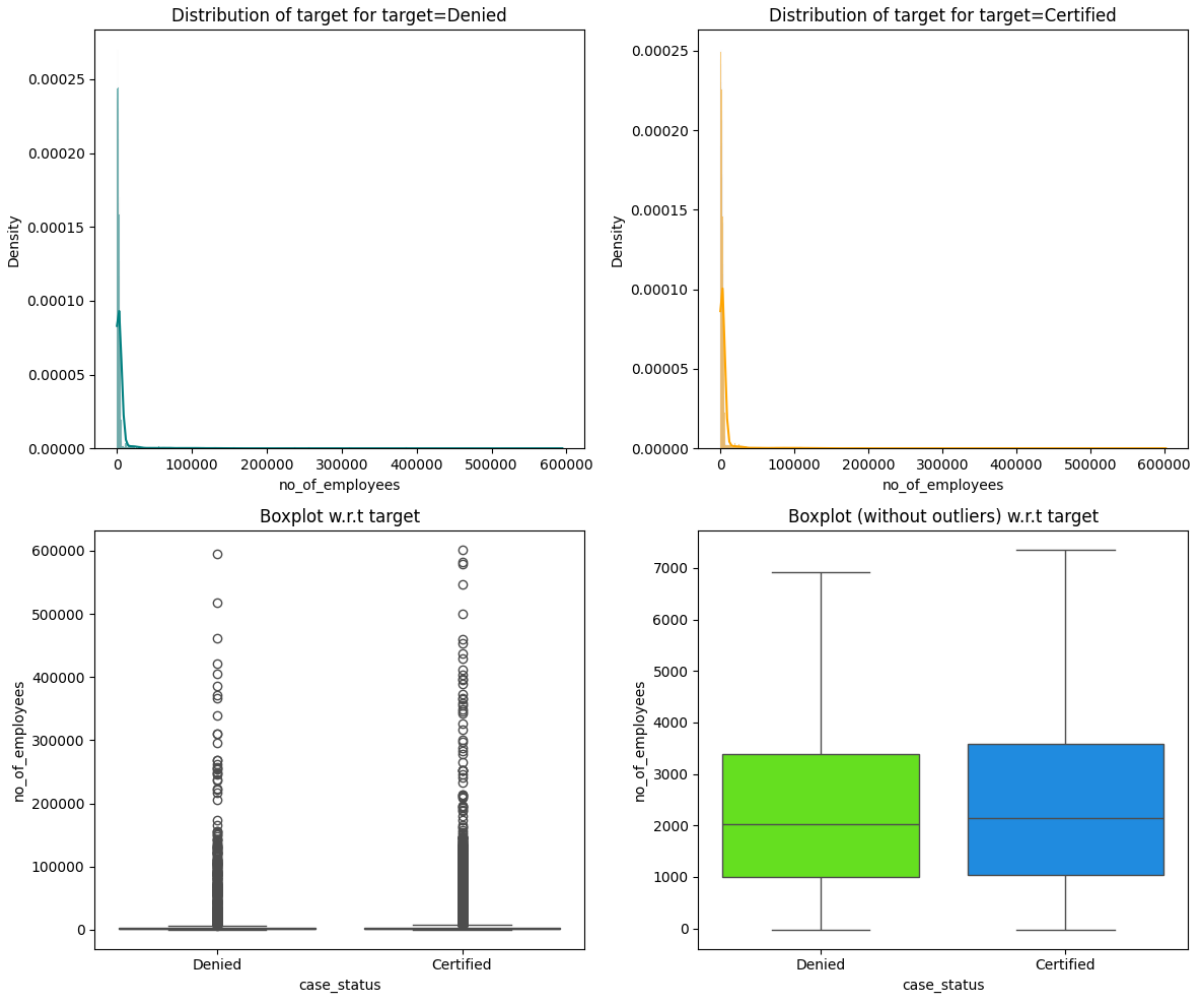

Figure 21 - Distribution plot between no_of_employees and case_study

- *Majority of employees have their visa applications certified when compared to denied applications.*

Heatmap

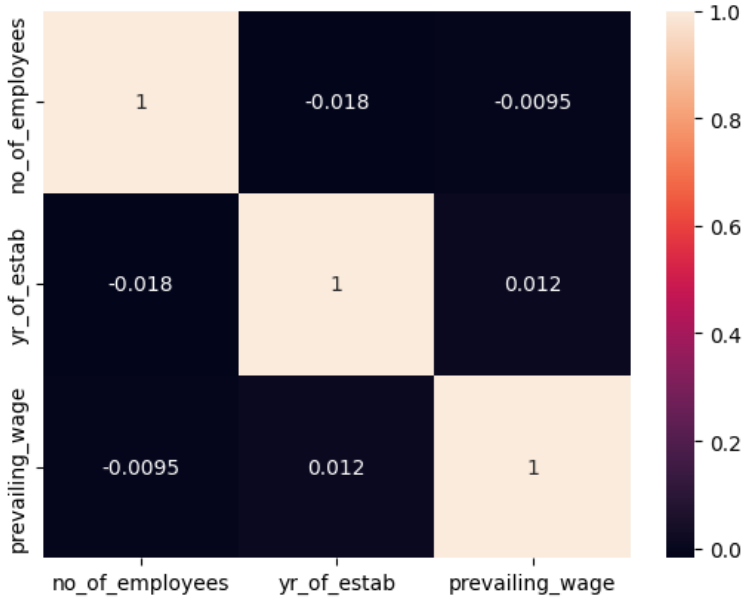

Figure 22 - Heatmap of numerical columns

- *we can see there is no strong correlation between numerical columns.*

Data Preprocessing

Missing value Treatment

	0
case_id	0
continent	0
education_of_employee	0
has_job_experience	0
requires_job_training	0
no_of_employees	0
yr_of_estab	0
region_of_employment	0
prevailing_wage	0
unit_of_wage	0
full_time_position	0
case_status	0

Figure 23 - Null values count

```
df.duplicated().sum()

np.int64(0)
```

Figure 24 - Duplicate value count

- *There are no null values and duplicate values in the data.*

Outliers Check

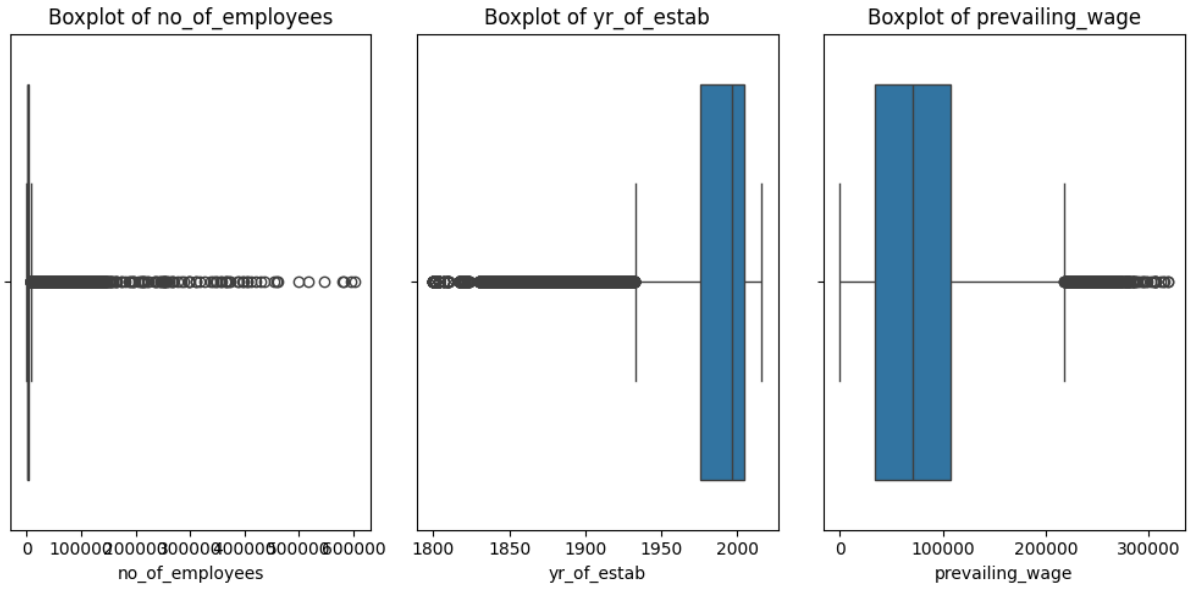

Figure 25 - Outliers of columns

- *There are many numerical outliers in each numerical column but we cannot treat them because they are significant values that influence the target variable.*

Data Preparation for modelling

- *We can drop case_id column as it does not contribute for prediction.*
- *we cannot have **negative values** in no_of_employees column so we have to **replace negative values with np.nan and impute the negative values with median of values.***
- *We replace the categorical values in **'has_job_experience' , 'requires_job_training' , 'full_time_position' , 'caste_status'** columns into numerical values.*
- *We create dummies to **'continent', 'education_of_employee' , 'region_of_employment' , ' unit_of_wage'** columns and drop the first value.*

	has_job_experience	requires_job_training	no_of_employees	yr_of_estab	prevailing_wage	full_time_position	case_status	continent_Africa	continent_Asia
0	0	0	14513.0	2007	592.2029	1	0	0.0	1.0
1	1	0	2412.0	2002	83425.6500	1	1	0.0	1.0
2	0	1	44444.0	2008	122996.8600	1	0	0.0	1.0
3	0	0	98.0	1897	83434.0300	1	0	0.0	1.0
4	1	0	1082.0	2005	149907.3900	1	1	1.0	0.0
...	...	...	...	...	...	...	...	...	...
25475	1	1	2601.0	2008	77092.5700	1	1	0.0	1.0
25476	1	0	3274.0	2006	279174.7900	1	1	0.0	1.0
25477	1	0	1121.0	1910	146298.8500	0	1	0.0	1.0
25478	1	1	1918.0	1887	86154.7700	1	1	0.0	1.0
25479	1	0	3195.0	1960	70876.9100	1	1	0.0	1.0

25480 rows × 26 columns

Figure 26 - Data after pre-processing

After Pre-Processing:

- *No of Rows = 25480*
- *No of Columns = 26*

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25480 entries, 0 to 25479
Data columns (total 26 columns):
 #   Column                                 Non-Null Count  Dtype
---  ------                                 --------------  -----
 0   has_job_experience                     25480 non-null  int64
 1   requires_job_training                  25480 non-null  int64
 2   no_of_employees                        25480 non-null  float64
 3   yr_of_estab                            25480 non-null  int64
 4   prevailing_wage                        25480 non-null  float64
 5   full_time_position                     25480 non-null  int64
 6   case_status                            25480 non-null  int64
 7   continent_Africa                       25480 non-null  float64
 8   continent_Asia                         25480 non-null  float64
 9   continent_Europe                       25480 non-null  float64
 10  continent_North America                25480 non-null  float64
 11  continent_Oceania                      25480 non-null  float64
 12  continent_South America                25480 non-null  float64
 13  education_of_employee_Bachelor's       25480 non-null  float64
 14  education_of_employee_Doctorate        25480 non-null  float64
 15  education_of_employee_High School      25480 non-null  float64
 16  education_of_employee_Master's         25480 non-null  float64
 17  region_of_employment_Island            25480 non-null  float64
 18  region_of_employment_Midwest           25480 non-null  float64
 19  region_of_employment_Northeast         25480 non-null  float64
 20  region_of_employment_South             25480 non-null  float64
 21  region_of_employment_West              25480 non-null  float64
 22  unit_of_wage_Hour                      25480 non-null  float64
 23  unit_of_wage_Month                     25480 non-null  float64
 24  unit_of_wage_Week                      25480 non-null  float64
 25  unit_of_wage_Year                      25480 non-null  float64
dtypes: float64(21), int64(5)
memory usage: 5.1 MB
```

Figure 27 - Information of Data after pre-processing

- *After pre-processing all the columns in the data are converted into numerical for training the model.*

Data Splitting

```
Number of rows and columns in train data = (14268, 25)
Number of rows and columns in validation data = (6116, 25)
Number of rows and columns in test data = (5096, 25)
```

Figure 28 - Train-Test-Split

- *The image shows how the data is split into training set, validation set and testing set.*

Model Building

- *Recall score is considered as the appropriate metric for model evaluation.*

Model Building - Original Data

We are going to build 5 different models on the original dataset.

1. *Decision Tree Model with Bagging classifier*
2. *Random forest Model*
3. *Gradient Boosting Model*
4. *Adaptive Boosting Model*
5. *Decision Tree Model*

```
Training Performance:

Bagging: 0.9845734074929163
Random forest: 1.0
GBM: 0.8834085423444223
Adaboost: 0.8919089096442439
dtree: 1.0
--------------------------------------
Validation Performance:

Bagging: 0.7858017135862914
Random forest: 0.8440636474908201
GBM: 0.8734394124847001
Adaboost: 0.8849449204406364
dtree: 0.7571603427172583
```

Figure 29 - Recall Scores of 5 different model performance on original Data

```
Training and Validation Performance Difference:

Bagging: Training Score: 0.9846, Validation Score: 0.7858, Difference: 0.1988
Random forest: Training Score: 1.0000, Validation Score: 0.8441, Difference: 0.1559
GBM: Training Score: 0.8834, Validation Score: 0.8734, Difference: 0.0100
Adaboost: Training Score: 0.8919, Validation Score: 0.8849, Difference: 0.0070
dtree: Training Score: 1.0000, Validation Score: 0.7572, Difference: 0.2428
```

Figure 30 - Performance difference of model on Training data and Validation data

- ***Adaboost** model has the best performance followed by **GBM** model as per the **validation performance**.*

Model Building - Oversampled Data

```
Before Oversampling, counts of Visa applications Certified: 9529
Before Oversampling, counts of Visa applications Denied: 4739

After Oversampling, counts of Visa applications Certified: 9529
After Oversampling, counts of Visa applications Denied: 9529

After Oversampling, the shape of X_train: (19058, 25)
After Oversampling, the shape of y_train: (19058,)
```

Figure 31 - Oversampling of data

```
Training Performance:

Bagging: 0.9831042082065274
Random forest: 1.0
GBM: 0.8586420400881519
Adaboost: 0.8547591562598383
dtree: 1.0
---------------------------------
Validation Performance:

Bagging: 0.7725826193390453
Random forest: 0.824969400244798
GBM: 0.8460220318237454
Adaboost: 0.8425948592411261
dtree: 0.7380660954712362
```

Figure 32 - Recall scores of 5 different model performance on oversampled data

```
Training and Validation Performance Difference:

Bagging: Training Score: 0.9831, Validation Score: 0.7726, Difference: 0.2105
Random forest: Training Score: 1.0000, Validation Score: 0.8250, Difference: 0.1750
GBM: Training Score: 0.8586, Validation Score: 0.8460, Difference: 0.0126
Adaboost: Training Score: 0.8548, Validation Score: 0.8426, Difference: 0.0122
dtree: Training Score: 1.0000, Validation Score: 0.7381, Difference: 0.2619
```

Figure 33 - Performance difference of Oversampled model on Training data and Validation data

- ***GBM** has the best performance followed by **AdaBoost** model as per the **validation performance.***

Model Building - Undersampled Data

```
Before Undersampling, counts of Visa applications Certified: 9529
Before Undersampling, counts of Visa applications Denied: 4739

After Undersampling, counts of Visa applications Certified: 4739
After Undersampling, counts of Visa applications Denied: 4739

After Undersampling, the shape of X_train: (9478, 25)
After Undersampling, the shape of y_train: (9478,)
```

Figure 34 - Under sampling of data

```
Training Performance:

Bagging: 0.9662376028698038
Random forest: 0.9997889850179362
GBM: 0.7387634522051065
Adaboost: 0.7113315045368221
dtree: 1.0
---------------------------------
Validation Performance:

Bagging: 0.617625458996328
Random forest: 0.6682986536107711
GBM: 0.7238678090575276
Adaboost: 0.7126070991432069
dtree: 0.616891064871481
```

Figure 35 - Recall scores of 5 different model performance on Undersampled data

```
Training and Validation Performance Difference:

Bagging: Training Score: 0.9662, Validation Score: 0.6176, Difference: 0.3486
Random forest: Training Score: 0.9998, Validation Score: 0.6683, Difference: 0.3315
GBM: Training Score: 0.7388, Validation Score: 0.7239, Difference: 0.0149
Adaboost: Training Score: 0.7113, Validation Score: 0.7126, Difference: -0.0013
dtree: Training Score: 1.0000, Validation Score: 0.6169, Difference: 0.3831
```

Figure 36 - Performance difference of Undersampled model on Training data and Validation data

- ***GBM** has the best performance followed by **AdaBoost** model as per the **validation performance**.*

- *After building 15 models, we observed that both the **GBM and Adaboost** models, trained on an undersampled and oversampled dataset shows good performance on both the training and validation datasets.*

Hyperparameter Tuning

Tuning AdaBoostClassifier model with Undersampled data

```
RandomizedSearchCV
RandomizedSearchCV(cv=5, estimator=AdaBoostClassifier(random_state=1),
                   n_iter=50, n_jobs=-1,
                   param_distributions={'estimator': [DecisionTreeClassifier(max_depth=1,
                                                                             random_state=1),
                                                      DecisionTreeClassifier(max_depth=2,
                                                                             random_state=1),
                                                      DecisionTreeClassifier(max_depth=3,
                                                                             random_state=1)],
                                        'learning_rate': [0.1, 0.01, 0.2, 0.05,
                                                          1],
                                        'n_estimators': array([10, 20, 30])},
best_estimator_: AdaBoostClassifier
AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1,
                                                    random_state=1),
                   learning_rate=0.1, n_estimators=np.int64(10),
                   random_state=1)
estimator: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=1, random_state=1)
DecisionTreeClassifier
```

Figure 37 - Hyper parameter tuning of Ada boost classifier with undersampled data

Finding Best parameters

```
Best parameters:
 {'n_estimators': np.int64(10), 'learning_rate': 0.1, 'estimator': DecisionTreeClassifier(max_depth=1, random_state=1)}
```

Figure 38 - Best parameters

Model tuning with best parameters

```
AdaBoostClassifier
AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1,
                                                    random_state=1),
                   learning_rate=0.1, n_estimators=10, random_state=1)
estimator: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=1, random_state=1)
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=1, random_state=1)
```

Figure 39 - Model with best parameters

Training performance of model with best parameters

	Accuracy	Recall	Precision	F1
0	0.600443	0.932053	0.560391	0.699945

Figure 40 - Performance matrices of training set

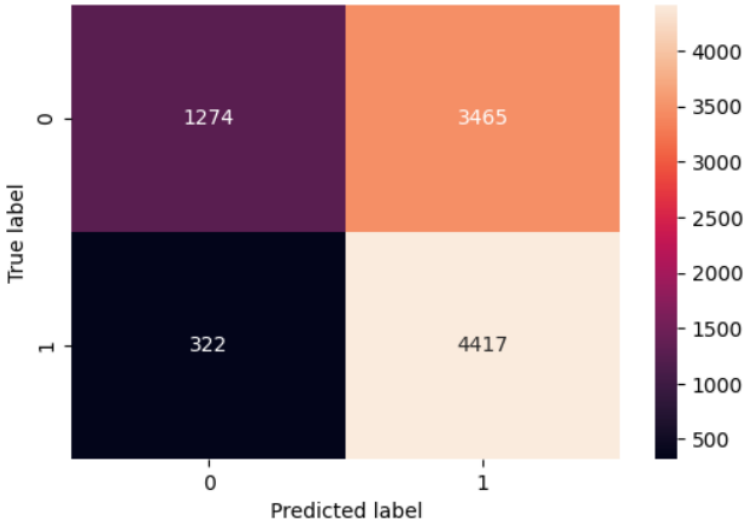

Figure 41 - confusion matrix of training set

- *Model performs well on training set with recall score of **93.2%.***

Validation performance of model with best parameters

Accuracy	Recall	Precision	F1
0.711086	0.932681	0.718597	0.811761

Figure 42 - Performance matrices of validation set

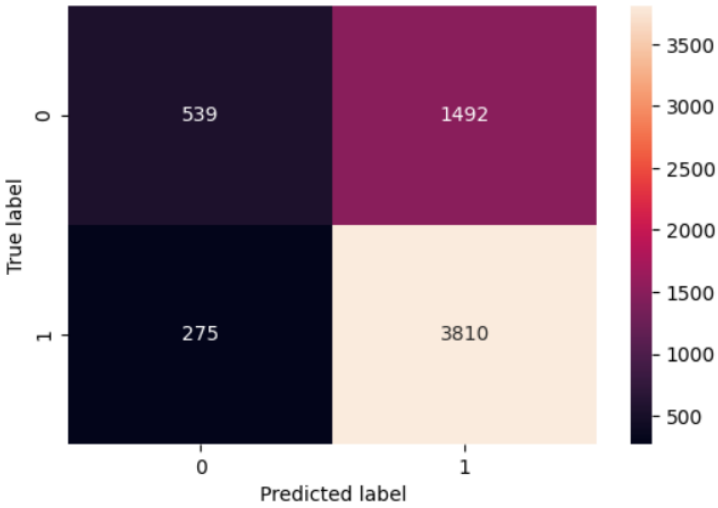

Figure 43 - Confusion matrix of validation set

- *Model performs well on validation set with recall score of **93.26%.***

Tuning Gradient Boosting model with Undersampled Data

```
RandomizedSearchCV
RandomizedSearchCV(cv=5, estimator=GradientBoostingClassifier(random_state=1),
                   n_iter=50, n_jobs=-1,
                   param_distributions={'init': [AdaBoostClassifier(random_state=1),
                                                 DecisionTreeClassifier(random_state=1)],
                                        'learning_rate': [0.01, 0.2, 0.05, 1],
                                        'max_features': [0.5, 0.7, 1],
                                        'n_estimators': array([125, 150]),
                                        'subsample': [0.8, 0.9, 1]},
                   random_state=1,
                   scoring=make_scorer(recall_score, response_method='predict'))
best_estimator_: GradientBoostingClassifier
GradientBoostingClassifier(init=AdaBoostClassifier(random_state=1),
                           learning_rate=0.01, max_features=0.5,
                           n_estimators=np.int64(150), random_state=1,
                           subsample=1)
init: AdaBoostClassifier
AdaBoostClassifier(random_state=1)
AdaBoostClassifier
```

Figure 44 - Hyper parameter tuning of Gradient boost classifier with undersampled data

Finding Best Parameters

```
Best parameters:
 {'subsample': 1, 'n_estimators': np.int64(150), 'max_features': 0.5, 'learning_rate': 0.01, 'init': AdaBoostClassifier(random_state
```

Figure 45 - Best parameters

Model tuning with best parameters

```
GradientBoostingClassifier
GradientBoostingClassifier(init=AdaBoostClassifier(random_state=1),
                           learning_rate=0.01, max_features=0.5,
                           n_estimators=150, random_state=1, subsample=1)
init: AdaBoostClassifier
AdaBoostClassifier(random_state=1)
AdaBoostClassifier
AdaBoostClassifier(random_state=1)
```

Figure 46 - Best Model

Training performance of model with best parameters

Accuracy	Recall	Precision	F1
0.708377	0.748048	0.69306	0.719505

Figure 47 - Performance metrices of training set

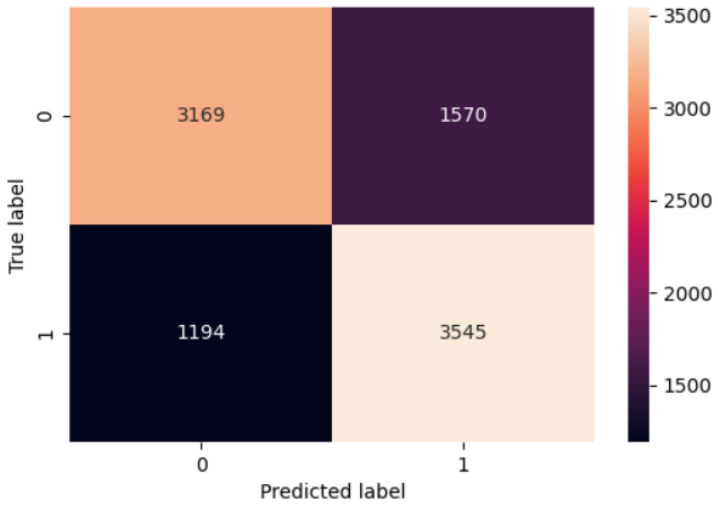

Figure 48 - confusion matrix of training set

- *Model performs decently on training set with recall score of **74.8%**.*

Validation performance of model with best parameters

Accuracy	Recall	Precision	F1
0.718934	0.744186	0.818525	0.779587

Figure 49 - Performance metrices of validation set

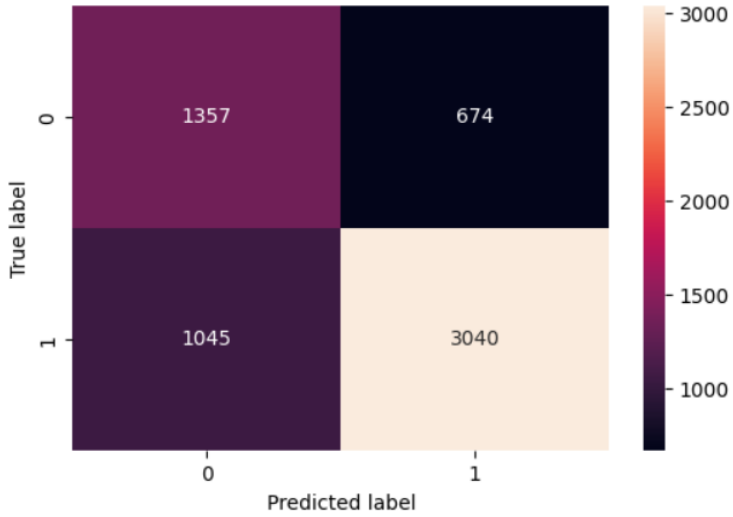

Figure 50 - confusion matrix of validation set

- *Model performs decently on validation set with recall score of **74.4%**.*

Tuning Adaboosting model with Oversampled data

```
RandomizedSearchCV
RandomizedSearchCV(cv=5, estimator=AdaBoostClassifier(random_state=1),
                   n_iter=50, n_jobs=-1,
                   param_distributions={'estimator': [DecisionTreeClassifier(max_depth=1,
                                                                             random_state=1),
                                                      DecisionTreeClassifier(max_depth=2,
                                                                             random_state=1),
                                                      DecisionTreeClassifier(max_depth=3,
                                                                             random_state=1),
                                                      DecisionTreeClassifier(max_depth=4,
                                                                             random_state=1),
                                                      DecisionTreeClassifier(max_depth=5,
best_estimator_: AdaBoostClassifier
AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=4,
                                                    random_state=1),
                   learning_rate=0.01, n_estimators=np.int64(40),
                   random_state=1)
estimator: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=1)
DecisionTreeClassifier
```

Figure 51 - Hyperparameter tuning of Ada boost model with Oversampled data

Finding Best Parameters

```
Best parameters:
 {'n_estimators': np.int64(40), 'learning_rate': 0.01, 'estimator': DecisionTreeClassifier(max_depth=4, random_state=1)}
```

Figure 52 - Best Parameters

Model tuning with best parameters

```
AdaBoostClassifier
AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=4,
                                                    random_state=1),
                   learning_rate=0.01, n_estimators=40, random_state=1)
estimator: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=1)
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=1)
```

Figure 53 - Best Model

Training performance of model with best parameters

Accuracy	Recall	Precision	F1
0.767184	0.846679	0.730532	0.784329

Figure 54 - Performance matrices of training set

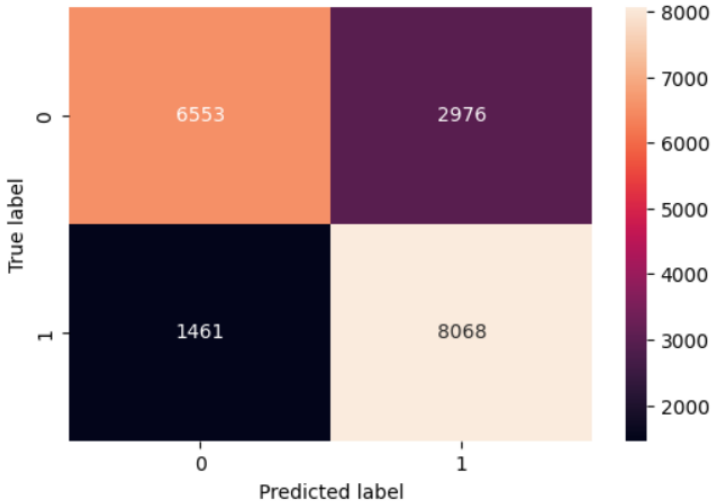

Figure 55 - confusion matrix of training set

- *Model performs well on training set with recall score of **84.66%.***

Validation performance of model with best parameters

Accuracy	Recall	Precision	F1
0.724984	0.840392	0.769214	0.803229

Figure 56 - Performance matrices of validation set

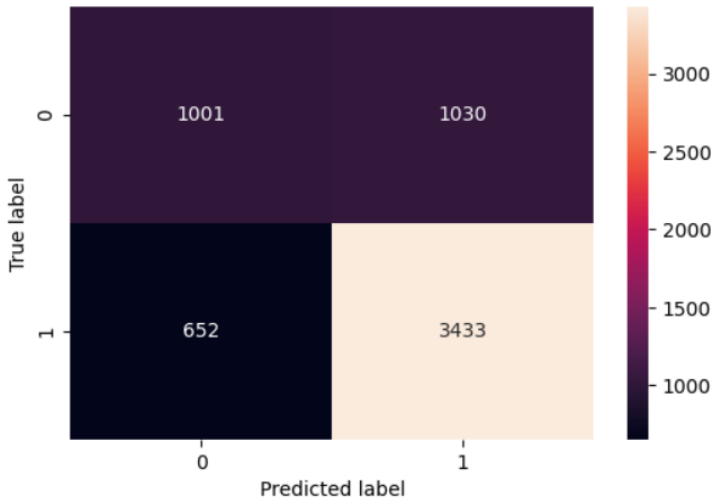

Figure 57 - confusion matrix of validation set

- *Model performs well on validation set with recall score of **84.03%.***

Tuning Gradient Boosting model with Oversampled data

```
RandomizedSearchCV
RandomizedSearchCV(cv=5, estimator=GradientBoostingClassifier(random_state=1),
                   n_iter=50, n_jobs=-1,
                   param_distributions={'init': [AdaBoostClassifier(random_state=1),
                                                 DecisionTreeClassifier(random_state=1)],
                                        'learning_rate': [0.1, 0.01, 0.2, 0.05,
                                                          1],
                                        'max_features': [0.5, 0.7, 1],
                                        'n_estimators': array([ 75, 100, 125]),
                                        'subsample': [0.5, 0.7, 1]},
                   random_state=1,
                   scoring=make_scorer(recall_score, response_method='predict'))
best_estimator_: GradientBoostingClassifier
GradientBoostingClassifier(init=AdaBoostClassifier(random_state=1),
                           learning_rate=0.01, max_features=1,
                           n_estimators=np.int64(125), random_state=1,
                           subsample=0.5)
init: AdaBoostClassifier
AdaBoostClassifier(random_state=1)
AdaBoostClassifier
AdaBoostClassifier(random_state=1)
```

Figure 58 - Hyperparameter tuning of Gradient boosting model with Oversampled Data

Finding Best Model

```
Best parameters:
 {'subsample': 0.5, 'n_estimators': np.int64(125), 'max_features': 1, 'learning_rate': 0.01, 'init': AdaBoostClassifier(random_state
```

Figure 59 - Best Parameters

Model tuning with best parameters

```
GradientBoostingClassifier
GradientBoostingClassifier(init=AdaBoostClassifier(random_state=1),
                           learning_rate=0.01, max_features=1, n_estimators=125,
                           random_state=1, subsample=0.5)
init: AdaBoostClassifier
AdaBoostClassifier(random_state=1)
AdaBoostClassifier
AdaBoostClassifier(random_state=1)
```

Figure 60 - Best Model

Training performance of model with best parameters

Accuracy	Recall	Precision	F1
0.796463	0.873859	0.756725	0.811085

Figure 61 - Performance matrices of training set

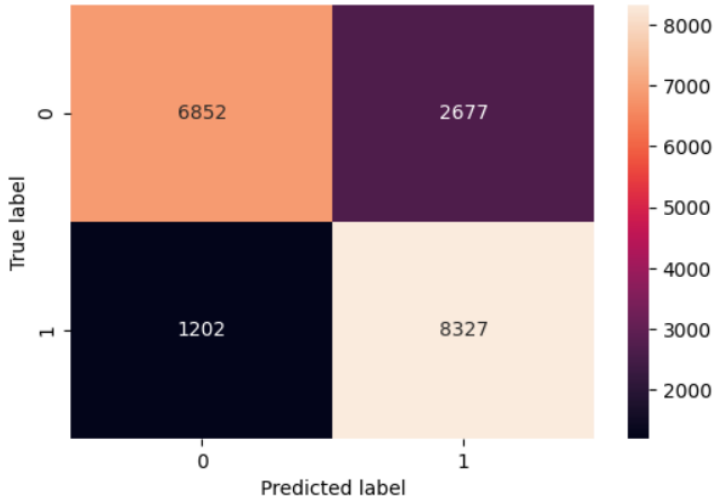

Figure 62 - confusion matrix of training set

- *Model performs well on training set with recall score of **87.38%.***

Validation performance of model with best parameters

Accuracy	Recall	Precision	F1
0.730216	0.864627	0.763016	0.81065

Figure 63 - Performance matrices of validation set

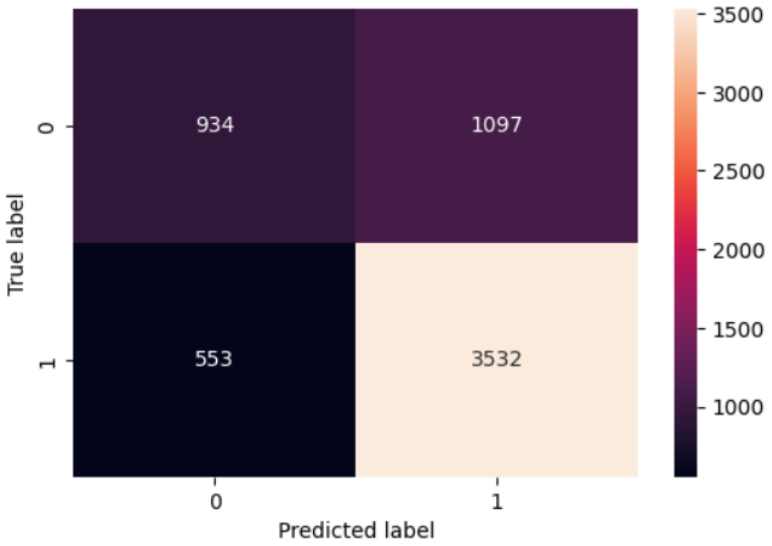

Figure 64 - confusion matrix of validation set

- *Model performs well on validation set with recall score of **86.46%.***

Model Comparison and Final Model Selection

Training performance comparison

Training performance comparison:

	AdaBoost trained with Undersampled data	Gradient boosting trained with Undersampled data	AdaBoost trained with Oversampled data	Gradient boosting trained with Oversampled data
Accuracy	0.600443	0.708377	0.767184	0.796463
Recall	0.932053	0.748048	0.846679	0.873859
Precision	0.560391	0.693060	0.730532	0.756725
F1	0.699945	0.719505	0.784329	0.811085

Figure 65 - Performance comparison of Training set

Validation performance comparison

Validation performance comparison:

	AdaBoost trained with Undersampled data	Gradient boosting trained with Undersampled data	AdaBoost trained with Oversampled data	Gradient boosting trained with Oversampled data
Accuracy	0.711086	0.718934	0.724984	0.730216
Recall	0.932681	0.744186	0.840392	0.864627
Precision	0.718597	0.818525	0.769214	0.763016
F1	0.811761	0.779587	0.803229	0.810650

Figure 66 - Performance comparison of Validation set

- ***AdaBoost model trained with under sampled data*** *has generalised performance, so let's consider it as the best model.*

Feature Importance

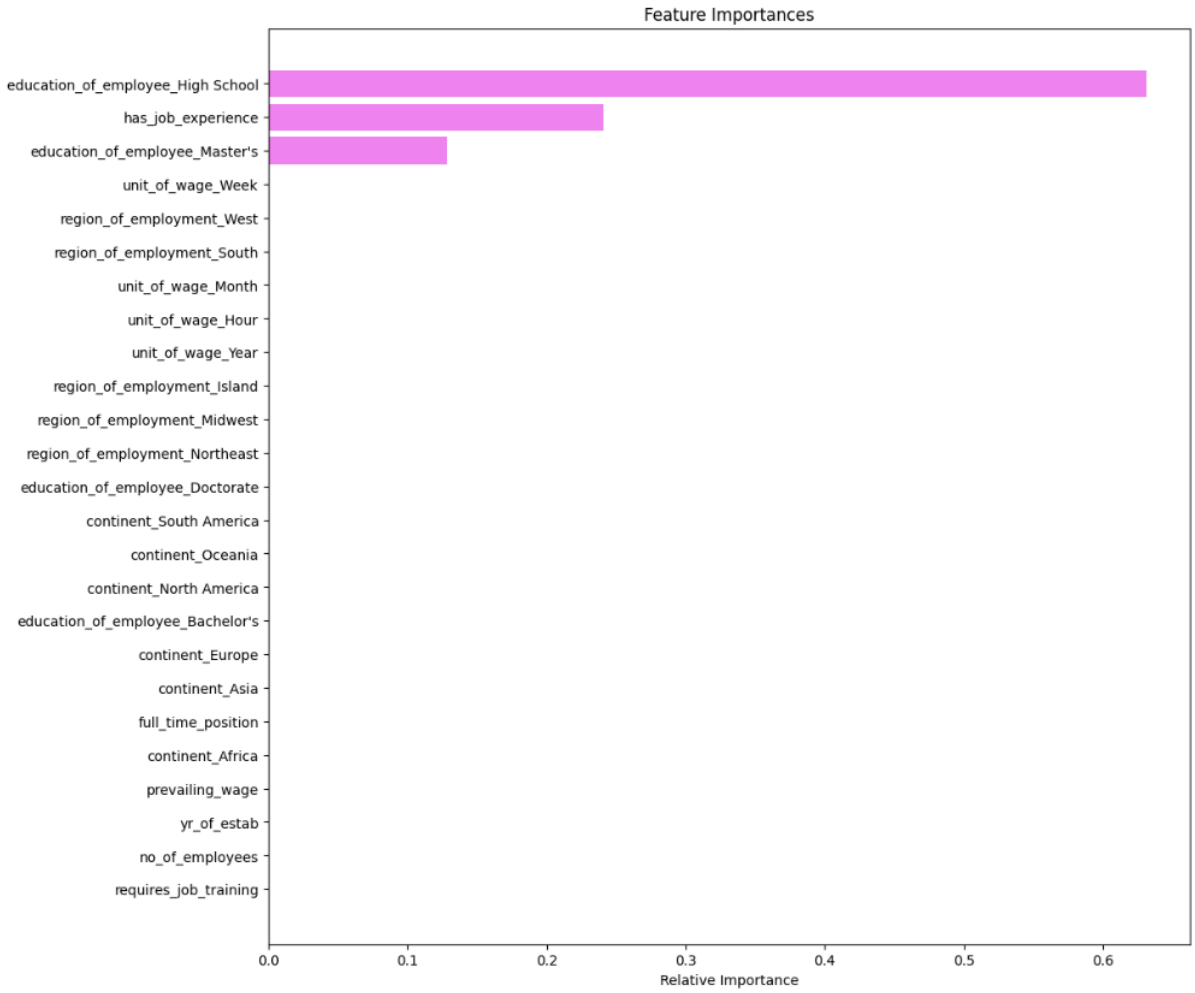

Figure 67 - Feature Importances on final model

- *We can see that **has_job_experience, education_of_employee_High School, education_of_employee_master's** are the most important features for making predictions.*

Performance on test set

Accuracy	Recall	Precision	F1
0.708203	0.930082	0.717101	0.809822

Figure 68 - Performance metrices of testing set

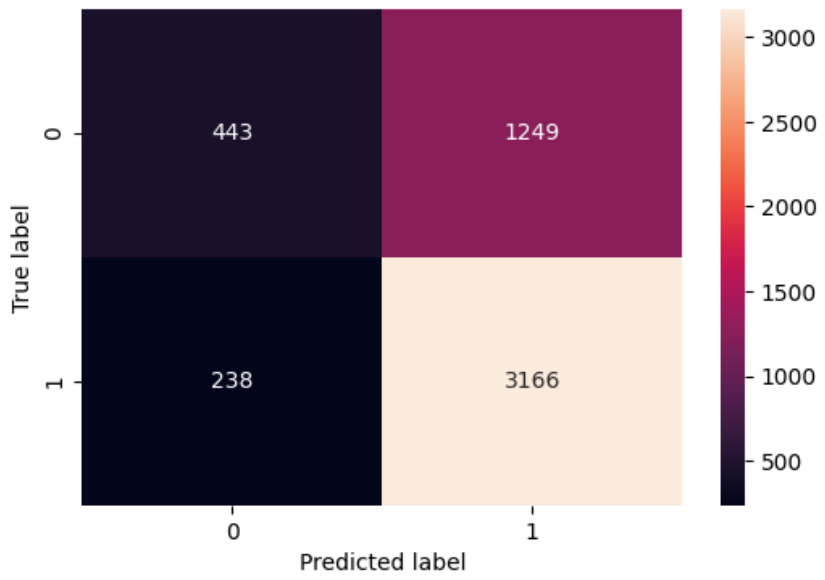

Figure 69 - Confusion matrix of testing set

- *The **Adaboost model trained on undersampled data has given ~76%** recall on the test set.*
- *So, this is a generalized model.*

Actionable Insights & Recommendations

- *Give more importance for hiring candidates with prior job experience and advanced degrees (Master's/Doctorate) to increase visa success rates.*
- *Regions like Midwest and Northeast regions are better for placements.*
- *Offer employees with yearly wage structures over hourly wage structures to reduce Visa rejection risks.*
- *As the recall score of AdaBoost model trained on undersampled data is higher use this model as the generalized model for better predictions.*
- *Choosing Asian regions over African regions is better as the visa denial rate is very less compared to African region.*